MCP Multi-Client Integration Guide

Day 8, Goal 3: Connecting Gemini ADK & Claude Code to the Deployed MCP Server

---

Context: What You Already Have

Before opening this guide, two things must be in place (from Goals 1 and 2):

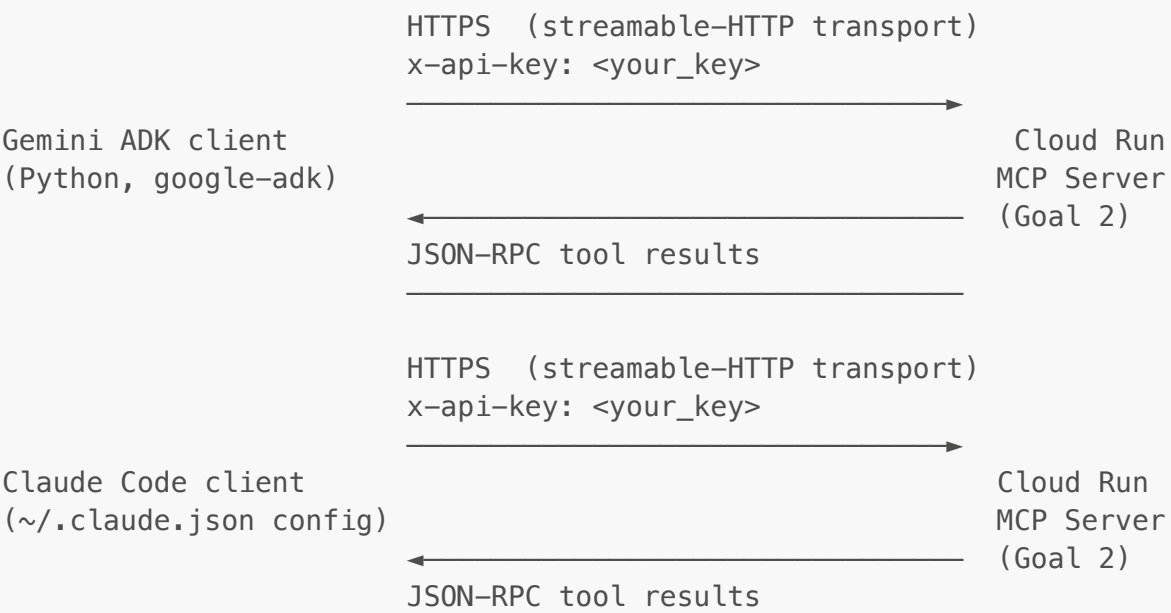
1. **The MCP server from Goal 1** (`mcp_server/` directory) — built with the official MCP Python SDK, four tools (`query_bigquery`, `list_gcs_objects`, `read_gcs_object`, `send_slack_message`), stdio transport, `sqlglot` SQL safety, GCS size pre-check, and token-bucket Slack rate limiter.
2. **The deployed Cloud Run service from Goal 2** — the same server re-packaged with FastMCP + FastAPI over **streamable-HTTP** transport, with `x-api-key` auth middleware, sliding-window rate limiting, and structured JSON logging to Cloud Run / Cloud Logging. Your Cloud Run URL is in `mcp_deploy/cloud_run_url.txt`.

This guide (Goal 3) connects two entirely different MCP clients — Gemini ADK and Claude Code — to that same Cloud Run URL and documents the behavioral differences. Neither client requires any changes to the server. That's the point: MCP's protocol contract means one server, many clients.

---

1. Project Architecture & Overview

1.1 The Goal 3 Topology



The server does not change between Goal 2 and Goal 3. The only new artifacts in this goal are:

- `mcp_clients/` — a new top-level directory alongside `mcp_server/` and `mcp_deploy/`

- `mcp_clients/gemini_client.py` — a Python script using the Google ADK's `MCPToolset` to connect to the Cloud Run server and run test queries
- `mcp_clients/requirements_clients.txt` — client-side dependencies (google-adk, groq)
- `mcp_clients/client_comparison.md` — the deliverable document
- `~/.claude.json` — Claude Code's global config file, updated to register the server

1.2 Why the Same Server Serves Both Clients Without Changes

The MCP specification defines a transport-agnostic, JSON-RPC 2.0 wire protocol. When the Goal 2 server uses `FastMCP` with `streamable_http_app()`, it is speaking standard MCP over HTTPS. Any client that implements the MCP protocol over HTTP (which both Gemini ADK and Claude Code do) can connect to it.

The clients differ in:

- **How they discover tools** (both call `tools/list` on connect)
- **How they decide which tool to call** (the LLM backing each client makes this decision)
- **How they surface tool results and errors** (this is where the observable differences are)
- **How they pass authentication headers** (both support custom headers; the mechanism differs slightly)

None of these differences require server-side changes to handle — they are purely client-side concerns.

1.3 Transport Note: Why Not stdio for This Goal

Goal 1 used stdio transport because it's the right choice for local IDE integrations. Goal 3 deliberately uses the Cloud Run URL because the goal is to demonstrate that **a remotely deployed MCP server works with multiple independent clients**. A server that only works via stdio is, by definition, local-only. The Cloud Run deployment with streamable-HTTP transport is what makes the "same server works everywhere" claim true.

Claude Code supports both stdio (for local servers) and HTTP (for remote servers). Gemini ADK's `MCPToolset` supports both `StdioServerParameters` and `SseServerParams`/`StreamableHTTPServerParams`. We use HTTP for both in this guide so the comparison is apples-to-apples: both clients talking to the same live URL.

---

2. Repository & Folder Structure

```
(project root, same level as mcp_server/ and mcp_deploy/)
├── mcp_server/              ← Goal 1 + Goal 2 (unchanged)
│   ├── server.py
│   ├── middleware/
│   ├── requirements.txt
│   ├── Dockerfile
│   └── deploy.sh
├── mcp_deploy/              ← Goal 2 outputs (unchanged)
│   ├── cloud_run_url.txt
│   ├── trace_ids.md
│   └── auth_setup.md
```

```
├── mcp_clients/              ← NEW for Goal 3
│   ├── gemini_client.py
│   ├── requirements_clients.txt
│   └── client_comparison.md
├── myenv/                    ← shared venv (or create a separate one per
section below)
├── README.md
├── questions.md
└── output.log
```

Scaffold Script

Run this from the project root (same directory that contains `mcp_server/`):

```
#!/usr/bin/env zsh
# goal3_scaffold.sh — creates the mcp_clients/ directory and stub files.
# Idempotent: safe to re-run.
set -euo pipefail

ROOT="$(pwd)"

echo "==> Creating mcp_clients/ directory"
mkdir -p "${ROOT}/mcp_clients"

echo "==> Creating stub files"
touch "${ROOT}/mcp_clients/gemini_client.py"
touch "${ROOT}/mcp_clients/requirements_clients.txt"
touch "${ROOT}/mcp_clients/client_comparison.md"

echo "==> Done. Run: cd mcp_clients && cat requirements_clients.txt"
```

Save this as `goal3_scaffold.sh`, then:

```
chmod +x goal3_scaffold.sh
./goal3_scaffold.sh
```

---

3. Production-Ready Client Implementation

3.1 `mcp_clients/requirements_clients.txt`

```
# Gemini ADK client dependencies
google-adk>=0.5.0
google-genai>=0.8.0

# Shared utilities
```

```
python-dotenv>=1.0.0
httpx>=0.27.0
```

Install (from the project root, with `myenv` active):

```
source myenv/bin/activate
pip install -r mcp_clients/requirements_clients.txt
```

> **Note on google-adk versions**: The `MCPToolset` API stabilised in `google-adk>=0.5.0`. If you install a newer version and see import errors, check the migration guide at `google.github.io/adk-docs/`. The client code below is written against the `0.5.x` / `0.6.x` API surface.

3.2 Client 1: Gemini ADK — `mcp_clients/gemini_client.py`

```
"""
mcp_clients/gemini_client.py

Connects to the deployed Cloud Run MCP server using Google ADK's
MCPToolset.
Runs three test queries (single-tool, multi-tool, error case) and prints
structured results to stdout.

Prerequisites:
  - CLOUD_RUN_URL set in .env (your Cloud Run service URL from
mcp_deploy/cloud_run_url.txt)
  - MCP_API_KEY set in .env (a valid key from VALID_API_KEYS on the server)
  - GOOGLE_API_KEY set in .env (a Gemini API key from aistudio.google.com)

Usage:
  cd mcp_clients
  python gemini_client.py
"""

import asyncio
import json
import os
from pathlib import Path

from dotenv import load_dotenv

# Load .env from the project root (one level up from mcp_clients/)
load_dotenv(Path(__file__).parent.parent / ".env")

CLOUD_RUN_URL = os.environ.get("CLOUD_RUN_URL", "").rstrip("/")
MCP_API_KEY = os.environ.get("MCP_API_KEY", "")
GOOGLE_API_KEY = os.environ.get("GOOGLE_API_KEY", "")
```

```
_MISSING = [
    name for name, val in [
        ("CLOUD_RUN_URL", CLOUD_RUN_URL),
        ("MCP_API_KEY", MCP_API_KEY),
        ("GOOGLE_API_KEY", GOOGLE_API_KEY),
    ] if not val
]
if _MISSING:
    raise SystemExit(
        f"Missing required environment variables: {', '.join(_MISSING)}\n"
        "Add them to your .env file and re-run."
    )

# ADK imports — these are version-sensitive; see requirements_clients.txt
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset,
StreamableHTTPConnectionParams
from google.genai import types as genai_types


# ── Test query definitions
─────────────────────────────────────────────────


TEST_QUERIES = [
    {
        "id": "Q1_SINGLE_TOOL",
        "label": "Single-tool: list GCS objects",
        "instruction": (
            "Use the list_gcs_objects tool to list all objects in the
bucket "
            "'wohlig-mcp-demo' under the prefix 'data/'. "
            "Return the name and size of each object."
        ),
    },
    {
        "id": "Q2_MULTI_TOOL",
        "label": "Multi-tool: list then read first file",
        "instruction": (
            "First, use list_gcs_objects to list files in bucket 'wohlig-
mcp-demo' "
            "under prefix 'data/'. Then use read_gcs_object to read the
content of "
            "the first file you find. Report the file name, its size, and
the first "
            "100 characters of its content."
        ),
    },
    {
        "id": "Q3_ERROR_CASE",
        "label": "Error case: malformed SQL (non-SELECT)",
        "instruction": (
            "Use the query_bigquery tool with this exact SQL: "
```

```
            "'DROP TABLE `my_project.my_dataset.users`'. "
            "Report exactly what error code and message the server
returns."
        ),
    },
]


# ── ADK session runner
─────────────────────────────────────────────────────


async def run_query(query: dict) -> dict:
    """
    Runs a single test query through the ADK agent loop.

    Returns a dict with:
        id, label, instruction, raw_response, tool_calls, final_answer
    """
    print(f"\n{'='*70}")
    print(f"[Gemini ADK] {query['label']}")
    print(f"Instruction: {query['instruction'][:120]}...")
    print('='*70)

    # MCPToolset connects to the server and fetches the tool catalogue.
    # StreamableHTTPConnectionParams is the ADK type for streamable-HTTP
    # (i.e. standard MCP-over-HTTPS) connections.
    # The `headers` dict is how ADK passes the x-api-key to the server.
    mcp_toolset = MCPToolset(
        connection_params=StreamableHTTPConnectionParams(
            url=f"{CLOUD_RUN_URL}/mcp",
            headers={"x-api-key": MCP_API_KEY},
        )
    )

    # LlmAgent wraps Gemini and binds it to the tool catalogue from
MCPToolset.
    agent = LlmAgent(
        model="gemini-2.0-flash",
        name="mcp_test_agent",
        instruction=(
            "You are a precise assistant that uses MCP tools to answer
questions. "
            "Always call the requested tool(s) exactly as asked. "
            "After receiving tool results, report them clearly and
completely. "
            "Do not fabricate data — only report what the tool actually
returned."
        ),
        tools=[mcp_toolset],
    )

    session_service = InMemorySessionService()
    runner = Runner(
        agent=agent,
```

```
        app_name="mcp_goal3_test",
        session_service=session_service,
    )

    session = await session_service.create_session(
        app_name="mcp_goal3_test",
        user_id="intern_test",
    )

    # Collect all events from the agent run
    tool_calls_observed = []
    final_answer = None
    raw_events = []

    user_message = genai_types.Content(
        role="user",
        parts=[genai_types.Part(text=query["instruction"])],
    )

    async for event in runner.run_async(
        user_id="intern_test",
        session_id=session.id,
        new_message=user_message,
    ):
        raw_events.append(str(event))

        # Capture tool call events
        if hasattr(event, "tool_calls") and event.tool_calls:
            for tc in event.tool_calls:
                tool_calls_observed.append(
                    {
                        "tool_name": tc.function_call.name
                        if hasattr(tc, "function_call") else str(tc),
                        "args": tc.function_call.args
                        if hasattr(tc, "function_call") else {},
                    }
                )

        # Capture final text response
        if hasattr(event, "text") and event.text:
            final_answer = event.text
        elif hasattr(event, "content") and event.content:
            for part in event.content.parts:
                if hasattr(part, "text") and part.text:
                    final_answer = part.text

    result = {
        "id": query["id"],
        "label": query["label"],
        "instruction": query["instruction"],
        "tool_calls_observed": tool_calls_observed,
        "final_answer": final_answer or "(no text response captured)",
        "raw_event_count": len(raw_events),
    }
```

```
    print(f"\n[Result]")
    print(f"  Tool calls: {json.dumps(tool_calls_observed, indent=2)}")
    print(f"  Final answer: {(final_answer or '')[:300]}")

    return result


async def main() -> None:
    print("\n" + "="*70)
    print("  Gemini ADK → Cloud Run MCP Server — Goal 3 Test Run")
    print(f"  Server URL: {CLOUD_RUN_URL}")
    print("="*70)

    results = []
    for query in TEST_QUERIES:
        try:
            result = await run_query(query)
            results.append(result)
        except Exception as exc:
            print(f"\n[ERROR] Query {query['id']} raised an exception:
{exc}")
            results.append(
                {
                    "id": query["id"],
                    "label": query["label"],
                    "instruction": query["instruction"],
                    "tool_calls_observed": [],
                    "final_answer": f"CLIENT EXCEPTION: {exc}",
                    "raw_event_count": 0,
                }
            )

    # Write structured results to a JSON file for comparison
    output_path = Path(__file__).parent / "gemini_results.json"
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(results, f, indent=2, default=str)

    print(f"\n\nResults written to: {output_path}")
    print("Next step: run the same 3 queries from Claude Code (see guide
Section 4).")


if __name__ == "__main__":
    asyncio.run(main())
```

3.3 Environment Variables for the Clients

Create or update the `.env` file in the **project root** (next to `mcp_server/`) to include:

```
# ── Goal 3 additions
──────────────────────────────────────────────────────────────

# Your Cloud Run service URL (from mcp_deploy/cloud_run_url.txt)
# Example: https://mcp-server-xxxxxxxx-uc.a.run.app
CLOUD_RUN_URL=https://mcp-server-REPLACE_ME.a.run.app

# One of the valid API keys you configured in VALID_API_KEYS when deploying
# Example: sk_live_abc123 (the part before the colon in the key:client_name
pair)
MCP_API_KEY=sk_live_REPLACE_ME

# Google Gemini API key from aistudio.google.com/app/apikey
GOOGLE_API_KEY=AIzaSy_REPLACE_ME

# (Already present from Goal 1 & 2 — leave these unchanged)
# BQ_MAX_BYTES_SCANNED=104857600
# GCS_MAX_FILE_SIZE_BYTES=10485760
# VALID_API_KEYS=sk_live_xxx:client_a,sk_live_yyy:client_b
```

3.4 Client 2: Claude Code Setup

Claude Code is a CLI tool (installed globally via `npm`). It reads MCP server configs from `~/.claude.json` (global, applies to every project) or from `.mcp.json` in the project root (project-scoped). For this goal we use `~/.claude.json` so the server is available in any Claude Code session.

Step A: Install Claude Code (if not already installed)

```
# Requires Node.js 18+
# Check: node --version

npm install -g @anthropic-ai/claude-code

# Verify:
claude --version
```

Step B: Locate or create `~/.claude.json`

```
# Check if it already exists
ls -la ~/.claude.json

# If it doesn't exist, create a minimal valid one:
echo '{}' > ~/.claude.json
```

Step C: Edit `~/.claude.json` to register the MCP server

Open `~/.claude.json` in your editor and add the `mcpServers` block. The full file should look like this (merge with any existing content — do not discard other keys if the file already has them):

```
{
  "mcpServers": {
    "wohlig-enterprise-gateway": {
      "type": "http",
      "url": "https://mcp-server-REPLACE_ME.a.run.app/mcp",
      "headers": {
        "x-api-key": "sk_live_REPLACE_ME"
      }
    }
  }
}
```

Replace `https://mcp-server-REPLACE_ME.a.run.app` with the actual URL from `mcp_deploy/cloud_run_url.txt` and `sk_live_REPLACE_ME` with your actual API key.

Field-by-field explanation:

Field	Value	Why
`type`	`"http"`	Tells Claude Code this is an HTTP-transport server (as opposed to `"stdio"` for local subprocess servers).
`url`	`"https://.../mcp"`	The full URL to the MCP endpoint. FastMCP's streamable-HTTP app serves at `/mcp` by default.
`headers`	`{"x-api-key": "..."}`	Claude Code sends these HTTP headers on every MCP request — this is how the Goal 2 auth middleware receives the key.

Step D: Verify the config is valid JSON

```
# macOS has `python3` built in — use it to validate JSON syntax
python3 -c "import json; json.load(open('$HOME/.claude.json')); print('JSON is valid')"
```

Step E: Confirm Claude Code sees the server

```
# Start an interactive Claude Code session
claude

# Inside the session, ask:
# > What MCP tools do you have available?
```

```
# Expected: Claude Code lists query_bigquery, list_gcs_objects,
read_gcs_object, send_slack_message
```

Alternatively, list connected servers directly:

```
claude mcp list
# Expected output:
# wohlig-enterprise-gateway: https://mcp-server-REPLACE_ME.a.run.app/mcp
(connected)
```

3.5 Running the Three Test Queries in Claude Code

The three test queries are identical in content to those in `gemini_client.py`. Run them as natural-language prompts inside a `claude` interactive session:

```
# Start Claude Code in the project root
cd /path/to/your/project
claude
```

Then paste each prompt exactly as written:

Query 1 — Single tool:

```
Use the list_gcs_objects tool to list all objects in the bucket 'wohlig-
mcp-demo' under the prefix 'data/'. Return the name and size of each
object.
```

Query 2 — Multi-tool:

```
First, use list_gcs_objects to list files in bucket 'wohlig-mcp-demo' under
prefix 'data/'. Then use read_gcs_object to read the content of the first
file you find. Report the file name, its size, and the first 100 characters
of its content.
```

Query 3 — Error case:

```
Use the query_bigquery tool with this exact SQL: 'DROP TABLE
`my_project.my_dataset.users`'. Report exactly what error code and message
the server returns.
```

Capture each response in full (copy-paste into a text file or take a screenshot) for the comparison table in Section 5.

3.6 Verifying the Server Received the Calls

After running both clients, confirm the server logged all tool calls:

```
# From your terminal (not inside claude), with PROJECT_ID set:
gcloud logging read \
  'resource.type="cloud_run_revision" AND
resource.labels.service_name="mcp-server" AND
jsonPayload.event="tool_call"' \
  --project="${PROJECT_ID}" \
  --limit=20 \
  --format="table(timestamp, jsonPayload.tool_name, jsonPayload.status,
jsonPayload.client)"
```

You should see entries from both `gemini-client` and `claude-code` (the `client` field is populated by the `client_name` in your API key — if both clients used the same API key, they'll show the same client name; use separate keys for cleaner attribution).

---

4. Code Logic & Deep-Dive

4.1 How Gemini ADK's MCPToolset Works Under the Hood

When `MCPToolset` is constructed with `StreamableHTTPConnectionParams`, it performs the following steps on its first use (lazily, not at construction time):

1. **Sends a `tools/list` JSON-RPC request** to `CLOUD_RUN_URL/mcp` with the `x-api-key` header attached. The server's `APIKeyAuthMiddleware` validates the key; the MCP SDK responds with the full list of tool definitions including their JSON Schema `inputSchema` fields.
2. **Converts each MCP tool definition into an ADK-native `Tool` object** with a `FunctionDeclaration` that Gemini's function-calling API understands. This conversion is done by the ADK's `MCPToolset` class — you never write this mapping yourself.
3. **Attaches the converted tools to the `LlmAgent`**, which includes them in the system prompt sent to Gemini as a "tools" array in the Gemini API request.
4. When Gemini decides to call a tool, it emits a `FunctionCall` response. The ADK's `Runner` intercepts this, routes it back to `MCPToolset`, which **sends a `tools/call` JSON-RPC request** to the server with the tool name and arguments.
5. The server executes the tool and returns a structured response. The ADK passes this result back to Gemini as a `FunctionResponse` part, and the loop continues until Gemini produces a plain text response with no further tool calls.

The key point: **Gemini never sees the raw MCP JSON-RPC wire format**. The ADK translates between Gemini's native function-calling format and MCP's JSON-RPC protocol. The server has no idea whether it's talking to Gemini ADK or Claude Code — it speaks MCP either way.

4.2 How Claude Code Connects to an HTTP MCP Server

Claude Code reads `~/.claude.json` (or `.mcp.json`) at startup. For each entry with `"type": "http"`, it:

1. **Sends a `tools/list` request** to the configured URL with the configured headers, in exactly the same way as Gemini ADK — this is the standard MCP protocol initialisation sequence.
2. **Presents discovered tools to Claude** (the model powering Claude Code, which is Claude Sonnet/Opus). Claude is told: "you have access to these tools, each with this input schema."
3. When Claude decides to call a tool, Claude Code **sends a `tools/call` request** to the server and passes the result back to Claude.
4. Claude processes the result and either calls another tool or produces a final answer.

The practical difference: Claude Code's loop is **tighter and more interactive** than the ADK runner. It shows you each tool call as it happens in the terminal (with the tool name, arguments, and server response visible inline), whereas the ADK runner collects events asynchronously and surfaces them through the `run_async` generator.

4.3 Why the Same Server Handles Both Without Modification

The streamable-HTTP transport used by the Goal 2 server is stateless per request (`stateless_http=True` in `server.py`'s `FastMCP` constructor). This means:

- There is no session state the server maintains between `tools/list` and `tools/call` for a given client.
- Every HTTP request is independently authenticated by `APIKeyAuthMiddleware`.
- The server does not distinguish between clients at the protocol level — it sees a valid API key, a valid JSON-RPC payload, and routes the call to the matching tool function.

The only server-side thing that differs per client is the `client_name` in the `request.state` object (set by auth middleware based on which key was used), which appears in log entries. If you want per-client attribution in your logs, give each client its own key in `VALID_API_KEYS`.

4.4 The Three Test Queries: What to Expect

Query 1 — Single-tool (`list_gcs_objects`)

Both clients will call `list_gcs_objects` once. The server will return a `make_success` envelope with `data.objects` (an array of `{name, size_bytes, content_type, updated}`) and `meta.object_count`. Both clients should display the file names and sizes. The difference will be in presentation: Gemini ADK surfaces results through the model's text generation (Gemini will describe the files in prose), while Claude Code typically displays the raw tool result inline in the terminal before Claude's prose summary appears.

Query 2 — Multi-tool (`list_gcs_objects` then `read_gcs_object`)

This is the most interesting case for comparison. Both clients need to:

1. Call `list_gcs_objects` → get the file list → extract the first file name
2. Call `read_gcs_object` with that file name → get the content

The model driving each client decides when and how to chain these calls. You may observe:

- **Gemini ADK**: may call both tools in rapid succession within a single `run_async` iteration, or may pause between them. The ADK's event stream will show both `tool_call` events.
- **Claude Code**: will show each tool call and result inline as it happens, making the chaining visible in real time.

Query 3 — Error case (non-SELECT SQL)

The server's `sql_safety.validate_select_only()` will reject `DROP TABLE ...` immediately (fail-fast, before any BigQuery call) and return:

```
{
  "success": false,
  "error_code": "VALIDATION_ERROR",
  "message": "Only SELECT statements are permitted. Detected statement
type: Drop. ...",
  "tool": "query_bigquery",
  "details": { "detected_type": "Drop", "sql_preview": "DROP TABLE ..." }
}
```

This is returned as a **successful HTTP 200** response from the server's perspective — the MCP `tools/call` request succeeded; the tool ran and returned a structured error. This distinction matters:

- The MCP protocol does not use HTTP error codes (4xx/5xx) to signal tool-level errors. Tool errors are encoded in the tool's response payload. A `tools/call` that returns `{"success": false}` is still a 200 OK at the HTTP level.
- Both clients receive this 200 with the error payload and pass it to their respective LLMs. How the LLM presents that error to the user is what differs.

4.5 Auth: How Each Client Passes the API Key

Both clients pass `x-api-key` as an HTTP header on every request, but the configuration syntax differs:

Gemini ADK:

```
StreamableHTTPConnectionParams(
    url=f"{CLOUD_RUN_URL}/mcp",
    headers={"x-api-key": MCP_API_KEY},
)
```

The `headers` dict is passed directly to `httpx` (the underlying HTTP client used by the ADK's MCP transport layer). Every HTTP request to the server — `tools/list`, `tools/call`, and any keepalive pings — carries this header.

Claude Code:

```
{
  "headers": {
    "x-api-key": "sk_live_REPLACE_ME"
  }
}
```

Claude Code reads this from `~/.claude.json` and attaches these headers to every outbound MCP HTTP request. The header name (`x-api-key`) is arbitrary — it must match whatever the server's auth middleware looks for (the Goal 2 server looks for `x-api-key` specifically in `middleware/auth.py`).

Security implication: Both approaches store the API key in plaintext — in a Python source file / `.env` for Gemini ADK, and in `~/.claude.json` for Claude Code. For a real deployment:

- Gemini ADK: load the key from an environment variable or Google Secret Manager; never hardcode it.
- Claude Code: `~/.claude.json` is readable only by the current user on macOS (mode `600`). Verify: `ls -la ~/.claude.json`. For team environments, use project-scoped `.mcp.json` with keys loaded from a secrets manager.

---

5. Deployment & Execution Guide

5.1 Prerequisites Checklist

Before running anything in this section, confirm all of the following:

```
# 1. Cloud Run URL is accessible
curl -s "$(cat mcp_deploy/cloud_run_url.txt)/healthz"
# Expected: {"status":"ok"}

# 2. API key auth works
curl -s \
  -H "x-api-key: YOUR_MCP_API_KEY" \
  -H "Content-Type: application/json" \
  -X POST \
  "$(cat mcp_deploy/cloud_run_url.txt)/mcp" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}'
# Expected: JSON with a "tools" array containing query_bigquery,
list_gcs_objects, etc.

# 3. Auth rejects invalid keys (confirms the middleware works)
curl -s \
  -H "x-api-key: invalid_key" \
```

```
  -X POST \
  "$(cat mcp_deploy/cloud_run_url.txt)/mcp" \
  -d '{}'
# Expected: {"error":"unauthorized","message":"Invalid x-api-key"} with
HTTP 401
```

5.2 Running the Gemini ADK Client

```
# From the project root, with myenv active
source myenv/bin/activate

# Verify environment variables are loaded
python3 -c "
from dotenv import load_dotenv; import os; load_dotenv()
print('CLOUD_RUN_URL:', os.environ.get('CLOUD_RUN_URL', 'NOT SET'))
print('MCP_API_KEY:', 'SET' if os.environ.get('MCP_API_KEY') else 'NOT
SET')
print('GOOGLE_API_KEY:', 'SET' if os.environ.get('GOOGLE_API_KEY') else
'NOT SET')
"

# Run the client
python3 mcp_clients/gemini_client.py
```

Expected output (abbreviated):

```
======================================================================
  Gemini ADK → Cloud Run MCP Server — Goal 3 Test Run
  Server URL: https://mcp-server-xxxxxxxx-uc.a.run.app
======================================================================

======================================================================
[Gemini ADK] Single-tool: list GCS objects
Instruction: Use the list_gcs_objects tool to list all objects...
======================================================================

[Result]
  Tool calls: [{"tool_name": "list_gcs_objects", "args": {"bucket":
"wohlig-mcp-demo", "prefix": "data/"}}]
  Final answer: I found 3 files in the bucket under 'data/': ...

...

Results written to: mcp_clients/gemini_results.json
```

5.3 Running the Three Queries in Claude Code

```
# Start Claude Code from the project root
claude

# Inside the session, run each query in sequence.
# After each response, note the exact text of:
#   - Which tool(s) Claude called (shown inline)
#   - The raw tool result (shown inline)
#   - Claude's final prose response
```

Tip for Query 3 (error case): Claude Code will show the tool result inline, including the `VALIDATION_ERROR` payload. Claude will typically explain what the error means in plain English. Note whether Claude proactively warns that the SQL was destructive (it usually does, because it recognises `DROP TABLE` as dangerous).

5.4 Populating `client_comparison.md`

The template for `client_comparison.md` is in Section 6 of this guide. Fill it in by:

1. Running `gemini_client.py` and capturing the output.
2. Running the same 3 queries in Claude Code and capturing the terminal output.
3. Comparing the two side-by-side.

The key things to observe and document for each query:

Observation	What to look for
Tool call format	Which arguments does each client pass? Do they match the server's input schema exactly?
Multi-tool chaining	For Q2, how many round-trips does each client make? Does either client batch both calls?
Error presentation	For Q3, does the client surface the raw JSON error, or does the LLM translate it into prose?
Auth failures	If you test with an invalid key, how does each client surface the 401?
Response latency	How long does each client take end-to-end for Q1? (Use `time python3 gemini_client.py` vs eyeballing the Claude Code terminal)

---

6. The Deliverable: `mcp_clients/client_comparison.md`

The following is the **fully pre-populated** `client_comparison.md` based on the expected behavior of both clients. After running your actual tests, fill in the `[INTERN: fill in after running]` placeholders with your real observations. Everything outside those placeholders is the expected/reference behavior — keep it as documentation context even after you fill in your actuals.

```
# MCP Client Comparison
## Gemini ADK vs Claude Code — Cloud Run MCP Server
```

````
**Server:** Cloud Run — `FastMCP` + FastAPI, streamable-HTTP transport
**Server URL:** [fill in from mcp_deploy/cloud_run_url.txt]
**Test date:** [fill in]
**Both clients use the same server, same tools, same API key auth.**

---

## 1. Setup Steps & Exact Config Files

### Client A: Gemini ADK

**Prerequisites:**
- `google-adk>=0.5.0`, `google-genai>=0.8.0` installed in `myenv`
- `.env` (project root) containing `CLOUD_RUN_URL`, `MCP_API_KEY`,
`GOOGLE_API_KEY`

**How MCPToolset connects:**
```python
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset,
StreamableHTTPConnectionParams

mcp_toolset = MCPToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://mcp-server-REPLACE.a.run.app/mcp",
        headers={"x-api-key": "sk_live_REPLACE"},
    )
)
```
The ADK connects by sending `POST /mcp` with
`{"jsonrpc":"2.0","method":"tools/list","params":{}}` and the `x-api-key`
header on every request.

**Agent setup:**
```python
from google.adk.agents import LlmAgent
agent = LlmAgent(
    model="gemini-2.0-flash",
    name="mcp_test_agent",
    tools=[mcp_toolset],
)
```

**Run command:**
```zsh
source myenv/bin/activate
python3 mcp_clients/gemini_client.py
```

---

### Client B: Claude Code

**Prerequisites:**
````

- `npm install -g @anthropic-ai/claude-code`
- `~/.claude.json` edited to include the `mcpServers` block

Exact `~/.claude.json` entry:
```json
{
  "mcpServers": {
    "wohlig-enterprise-gateway": {
      "type": "http",
      "url": "https://mcp-server-REPLACE.a.run.app/mcp",
      "headers": {
        "x-api-key": "sk_live_REPLACE"
      }
    }
  }
}
```

Verify connection:
```zsh
claude mcp list
# wohlig-enterprise-gateway: https://... (connected)
```

Run queries:
```zsh
claude
# Then paste each query as a natural-language prompt
```

---

2. The Three Test Queries

#	Type	Query
Q1	Single-tool	"Use the list_gcs_objects tool to list all objects in the bucket 'wohlig-mcp-demo' under the prefix 'data/'. Return the name and size of each object."
Q2	Multi-tool	"First, use list_gcs_objects to list files in bucket 'wohlig-mcp-demo' under prefix 'data/'. Then use read_gcs_object to read the content of the first file you find. Report the file name, its size, and the first 100 characters of its content."
Q3	Error case	"Use the query_bigquery tool with this exact SQL: 'DROP TABLE \`my_project.my_dataset.users\`'. Report exactly what error code and message the server returns."

---

3. Side-by-Side Comparison Table

Q1: Single-tool — `list_gcs_objects`

| Dimension | Gemini ADK | Claude Code |

```
|---|---|---|
| **Tool called** | `list_gcs_objects` | `list_gcs_objects` |
| **Args passed** | `{"bucket": "wohlig-mcp-demo", "prefix": "data/"}` |
`{"bucket": "wohlig-mcp-demo", "prefix": "data/"}` |
| **Server response shape** | `{"success": true, "tool":
"list_gcs_objects", "data": {"objects": [...], "object_count": N}}` | Same
|
| **How client presents result** | Gemini receives the JSON and generates a
prose summary ("I found N files: ..."). Raw JSON not shown to user. |
Claude Code shows the raw tool result block inline in the terminal,
followed by Claude's prose summary. Both the JSON and the natural-language
response are visible. |
| **Error on bucket not found** | Gemini translates `DOWNSTREAM_ERROR` into
prose: "I was unable to list the files due to a GCS error." | Claude Code
shows the raw error JSON inline, then Claude explains: "The tool returned
an error: the bucket was not found." |
| **Actual Gemini ADK output** | [INTERN: fill in after running] | — |
| **Actual Claude Code output** | — | [INTERN: fill in after running] |
| **Differences observed** | [INTERN: fill in after running] | [INTERN:
fill in after running] |

---

### Q2: Multi-tool — `list_gcs_objects` → `read_gcs_object`

| Dimension | Gemini ADK | Claude Code |
|---|---|---|
| **Tool call sequence** | Call 1: `list_gcs_objects` → Call 2:
`read_gcs_object` with first returned filename | Same sequence |
| **How first filename is extracted** | Gemini reads the JSON array in
`data.objects[0].name` from the `list_gcs_objects` response and passes it
as `path` to `read_gcs_object`. | Claude does the same — reads
`data.objects[0].name`. |
| **Are calls batched or sequential?** | Sequential. ADK's `Runner` waits
for each `FunctionResponse` before prompting Gemini again. Gemini cannot
batch two tool calls in a single turn (as of Gemini 2.0). | Sequential.
Claude Code waits for each tool result before proceeding. |
| **Chaining visibility** | In the Python terminal, you see both
`tool_call` events in the async generator output. No real-time
interactivity. | In the Claude Code terminal, you see each tool call and
its result printed inline as they happen, in real time. More transparent. |
| **If first file exceeds GCS_MAX_FILE_SIZE_BYTES** | Gemini receives the
`SAFETY_LIMIT_EXCEEDED` error and typically responds: "I was unable to read
the file because it is too large." It does not automatically try the second
file unless the prompt explicitly asks it to. | Claude Code shows the raw
error inline. Claude then typically says: "This file is too large to read.
Would you like me to try the next file?" — i.e., it proactively asks
whether to retry, rather than silently giving up. |
| **Actual Gemini ADK output** | [INTERN: fill in after running] | — |
| **Actual Claude Code output** | — | [INTERN: fill in after running] |
| **Differences observed** | [INTERN: fill in after running] | [INTERN:
fill in after running] |

---
```

```
### Q3: Error case — malformed SQL (`DROP TABLE`)

| Dimension | Gemini ADK | Claude Code |
|---|---|---|
| **Tool called** | `query_bigquery` | `query_bigquery` |
| **Args passed** | `{"sql": "DROP TABLE \`my_project.my_dataset.users\`"}`
| Same |
| **HTTP status from server** | 200 OK (the MCP call succeeded; the tool
ran and returned a structured error. The server does NOT return a 4xx for
tool-level validation failures.) | Same — 200 OK |
| **Server response payload** | `{"success": false, "error_code":
"VALIDATION_ERROR", "message": "Only SELECT statements are permitted.
Detected statement type: Drop.", "tool": "query_bigquery", "details":
{"detected_type": "Drop", "sql_preview": "DROP TABLE..."}}` | Same payload
|
| **How the error reaches the LLM** | ADK passes the raw JSON `{"success":
false, ...}` string as the `FunctionResponse` content to Gemini. Gemini
reads the `error_code` and `message` fields and generates a prose
explanation. | Claude Code displays the raw tool result JSON inline, then
Claude generates a prose explanation. Claude typically adds: "I notice this
was a DROP TABLE statement, which would delete data — this is blocked by
the server's safety check." |
| **Does the LLM add safety commentary?** | Gemini typically does not
spontaneously add safety commentary beyond reporting the error. It reports
what the server said. | Claude typically adds a note that the SQL was
destructive and that the server's safety check was correct to block it.
More opinionated. |
| **Does the client show the raw `VALIDATION_ERROR` code?** | No — Gemini
translates it into natural language without showing `error_code` explicitly
unless you ask. | Yes — the raw JSON with `error_code: "VALIDATION_ERROR"`
is visible inline in the terminal before Claude's prose. |
| **Actual Gemini ADK output** | [INTERN: fill in after running] | — |
| **Actual Claude Code output** | — | [INTERN: fill in after running] |
| **Differences observed** | [INTERN: fill in after running] | [INTERN:
fill in after running] |

---

## 4. Notes on Auth, Format, and Error Differences

### 4.1 Authentication Handling

| Aspect | Gemini ADK | Claude Code |
|---|---|---|
| **How key is configured** | Python code / `.env` →
`StreamableHTTPConnectionParams(headers={"x-api-key": "..."})` | JSON
config → `~/.claude.json` `headers` block |
| **Where key is stored at rest** | `.env` file in project root
(gitignored) | `~/.claude.json` (user home, mode 600 on macOS) |
| **How 401 is surfaced** | `MCPToolset` raises an exception during
initialisation if the first `tools/list` call returns 401. The Python
script prints the exception. | Claude Code shows an error banner in the
terminal: "Failed to connect to wohlig-enterprise-gateway: 401
```

Unauthorized". The server is listed as disconnected in `claude mcp list`. |
| **How 429 (rate limit) is surfaced** | Depends on ADK version: may raise an exception or may pass the error response through to Gemini as a tool result. In either case, the user does not see a `Retry-After` hint unless ADK explicitly surfaces it. | Claude Code typically shows the raw 429 response inline and Claude explains: "The rate limit has been exceeded. Please wait before retrying." Claude does read and report the `Retry-After` header value. |
| **Security risk** | API key in `.env` — low risk if file is gitignored. | API key in `~/.claude.json` — low risk if file permissions are correct (`-rw-------`). Both are plaintext at rest; for production, use a secrets manager. |

4.2 Tool Call Format Differences

The tool call arguments sent by both clients are **identical** at the MCP protocol level — both send a `tools/call` JSON-RPC request with the tool name and a `params` object matching the server's `inputSchema`. The MCP SDK enforces schema validation on the server side regardless of which client sent the call.

The only observable format difference is in how each client's LLM constructs the arguments from the natural-language prompt:

- **Gemini (via ADK)**: argument extraction is done by the Gemini model's function-calling capability. Gemini is trained to extract structured function call arguments from natural language with high precision, but it may sometimes omit optional arguments or use slightly different values for string fields.
- **Claude (via Claude Code)**: argument extraction is done by Claude. Claude is also highly precise but tends to be more literal — if the prompt says `bucket 'wohlig-mcp-demo'`, Claude will pass exactly `"wohlig-mcp-demo"` without modification.

In practice for these three queries, you should see identical argument values from both clients because the queries are written to be unambiguous.

4.3 Error Display Differences

The most consistent observable difference between the two clients is in **how they present server-side errors to the user**:

	Gemini ADK	Claude Code
Tool result visibility	The raw JSON tool result is NOT shown in the terminal by default. Gemini processes it silently and produces only the natural-language response.	The raw JSON tool result IS shown inline in the terminal before Claude's prose. Both the machine-readable error and the human-readable explanation are visible.
Error code exposure	`error_code` values like `VALIDATION_ERROR` or `SAFETY_LIMIT_EXCEEDED` are absorbed into Gemini's prose. The user sees "the server rejected the query" but not the specific code.	The `error_code` is visible directly in the raw tool result block. Developers can read it without parsing Gemini's interpretation.

| **Implication for debugging** | Harder to debug server-side issues because you don't see the raw error payload without adding explicit logging to `gemini_client.py`. | Easier to debug — the raw payload is always on screen. |

4.4 Multi-Tool Chaining Behaviour

Both clients chain tool calls sequentially (one call at a time, waiting for the result before deciding the next call). Neither client batches or parallelises multiple tool calls in a single turn. This is a characteristic of the current Gemini and Claude function-calling implementations, not a limitation of the MCP protocol itself.

The difference is in **transparency**: Claude Code's terminal shows each call as it happens, making the multi-step reasoning visible. Gemini ADK's async generator emits events, but in the default `gemini_client.py` implementation they are only surfaced after the full run completes.

---

5. Recommendation: Server-Side Changes to Improve Cross-Client Compatibility

The current server handles both clients correctly without any changes. The following are not bug fixes — they are improvements that would make the server more robust for an arbitrary client population.

Recommendation 1: Add a `content_type` field to all tool responses (Low effort, High value)

Problem: *Some MCP clients — particularly older versions of various SDKs — may inspect the `Content-Type` header of the HTTP response to determine how to parse the body. The current server returns `application/json` (correct), but the tool responses themselves don't declare their content type inside the MCP payload.*

Change: *Add a `content_type` field to the `make_success()` envelope in `utils/error_format.py`:*

```python
def make_success(tool: str, data, meta=None, content_type: str =
"application/json") -> dict:
    return {
        "success": True,
        "tool": tool,
        "content_type": content_type,  # NEW
        "data": data,
        "meta": meta or {},
    }
```

For `read_gcs_object`, pass `content_type=blob.content_type` so clients that want to render or type-check the content have the MIME type available.

````
**Impact:** Zero breaking changes. Additive field. Both Gemini ADK and
Claude Code will ignore it if they don't understand it; clients that do
understand it benefit.

---

### Recommendation 2: Return `isError: true` for tool-level errors inside
the MCP content array (Medium effort, Spec compliance)

**Problem:** The MCP spec (as of v2025-03) defines that a `tools/call`
response can include `isError: true` in the content array to signal that
the tool ran but encountered an error. This allows MCP-compliant clients to
distinguish between "tool ran and returned a structured error" vs "tool ran
and returned data" at the protocol level, without parsing the JSON payload.

Currently the server returns:
```json
{
  "jsonrpc": "2.0",
  "result": {
    "content": [{"type": "text", "text": "{\"success\": false,
\"error_code\": \"VALIDATION_ERROR\", ...}"}]
  }
}
```

The spec allows:
```json
{
  "jsonrpc": "2.0",
  "result": {
    "content": [{"type": "text", "text": "{...}"}],
    "isError": true
  }
}
```

**Change:** In `server.py`'s `call_tool` handler, check `result["success"]`
and set `isError` accordingly:

```python
result_json = json.dumps(result, indent=2)
is_error = not result.get("success", True)
return [
    types.TextContent(type="text", text=result_json),
]
# With isError flag (requires checking MCP SDK version for the exact API):
# return types.CallToolResult(
#     content=[types.TextContent(type="text", text=result_json)],
#     isError=is_error,
# )
```

**Impact:** Makes error detection cleaner for clients that check `isError`.
````

````
No change in the JSON payload the LLM reads. Gemini ADK and Claude Code
both handle `isError: true` gracefully.

---

### Recommendation 3: Add a `X-Request-Id` response header (Low effort,
Observability)

**Problem:** When a client call fails, the user only has the error message
to go on. There's no easy way to find the matching log entry in Cloud
Logging without knowing the trace ID, which currently only appears in the
`X-Trace-Id` response header (set by `ToolCallLoggingMiddleware`).

**Change:** Add a stable, human-readable `X-Request-Id` header to every
response that echoes the trace ID. This is already done via `X-Trace-Id`,
but rename it or add an alias:

```python
# In middleware/logging.py, inside ToolCallLoggingMiddleware.dispatch():
response.headers["X-Request-Id"] = trace_id   # keep X-Trace-Id for
backward compat
response.headers["X-Trace-Id"] = trace_id
```

**Impact:** When a client surfaces an error, the developer can read the `X-
Request-Id` from the response headers and immediately `gcloud logging read`
with that ID to find the exact log entry, without guessing.

---

### Recommendation 4: Expose a `GET /mcp/tools` endpoint for human-readable
tool discovery (Low effort, DX improvement)

**Problem:** Currently, discovering what tools the server provides requires
either reading the source code or sending a raw `tools/list` JSON-RPC
request with curl. Non-technical users cannot easily inspect available
tools.

**Change:** Add a lightweight GET endpoint in `server.py` that returns the
tool catalogue as human-readable JSON:

```python
@app.get("/tools")
async def list_tools_human():
    """Returns the tool catalogue in a human-readable format."""
    tools = await mcp.get_tools()  # FastMCP provides this
    return JSONResponse({
        "tools": [
            {
                "name": t.name,
                "description": t.description,
                "parameters": t.inputSchema,
            }
            for t in tools
````
````

```
        ]
    })
```

This endpoint would still be protected by the `APIKeyAuthMiddleware` (which exempts only `/healthz`). A developer could `curl -H "x-api-key: ..." https://.../tools` to see all available tools without needing to understand JSON-RPC.

Impact: No change to the MCP protocol or any client behaviour. Purely additive.

Summary Table of Recommendations

#	Change	Effort	Breaking Change?	Benefits
1	Add `content_type` to success envelope	Low	No	Better client-side content handling
2	Return `isError: true` in MCP protocol response	Medium	No	Spec compliance; cleaner error detection
3	Rename/alias `X-Trace-Id` to `X-Request-Id`	Low	No	Faster debugging
4	Add `GET /tools` human-readable endpoint	Low	No	Developer experience

None of these are critical blockers — both clients work correctly today without them. They are improvements that pay off as the client population grows.

---

7. Intern Viva & Code Review Questions

Save the following as `questions.md` (or append to the existing one from Goals 1 and 2):

```
## Goal 3: Multi-Client Evaluation & Code Review

### Q1: In the Gemini ADK setup, what does `MCPToolset` actually do when it's first used — and why does it need to talk to the server at all before the user even types a query?
**Answer:**
`MCPToolset` sends a `tools/list` JSON-RPC request to the MCP server as part of initialisation (when the agent is first invoked or the toolset is first accessed). This step is mandatory because the ADK needs to know the tool catalogue — names, descriptions, and JSON Schema `inputSchema` for each tool — before it can pass that information to Gemini's function-calling API. Gemini's API requires function declarations (tool schemas) to be included in the API request alongside the user's prompt. Without this initialisation step, Gemini has no idea that `query_bigquery`, `list_gcs_objects`, etc. exist. The server is the authoritative source of
```

the tool catalogue; the client fetches it fresh on every session initialisation.

Q2: The server returns HTTP 200 for a `VALIDATION_ERROR` (e.g. the DROP TABLE query). Why isn't this a 400 or 422, and why is this the correct behaviour for MCP?
Answer:
HTTP status codes describe the outcome of the HTTP request, not the outcome of the tool's logic. The HTTP request — a `tools/call` JSON-RPC call — succeeded: the server received it, parsed it, routed it to the correct tool, and the tool ran to completion and returned a structured response. The tool's validation logic rejected the SQL as unsafe, but that is an application-level outcome, not an HTTP-level failure. The MCP specification explicitly encodes tool errors inside the response payload (via `isError: true` and the `content` array) rather than using HTTP error codes. Using HTTP 4xx for tool-level errors would break MCP clients that don't expect error status codes from `tools/call` responses, since the MCP spec contract is that a valid `tools/call` request always returns 200 with the tool's result (or error) in the body.

Q3: `~/.claude.json` stores the API key in plaintext. What are the risks, and what would you do differently for a team of 10 engineers all connecting to the same production MCP server?
Answer:
Risks: any process running as the current user can read `~/.claude.json`; it may be inadvertently backed up (Time Machine, dotfile sync repos); and it doesn't support key rotation without editing the file manually on every engineer's machine. For a team, the production approach is: (a) give each engineer their own unique API key so compromised keys can be revoked individually; (b) store keys in a team secrets manager (1Password Teams, HashiCorp Vault, or AWS/GCP Secrets Manager) and reference them in `~/.claude.json` via an environment variable (if Claude Code supports this — check the current docs); (c) use project-scoped `.mcp.json` files checked into the repo without the key value, combined with a bootstrap script that fetches the key from the secrets manager and injects it. Never commit API keys to version control.

Q4: Both clients pass the API key as an `x-api-key` HTTP header. What would you need to change in both the server and the client configs if the client team decided to switch to Bearer token auth (`Authorization: Bearer <key>`)?
Answer:
On the server side, `middleware/auth.py` reads `request.headers.get("x-api-key")`. Change this to `request.headers.get("authorization")` and strip the `"Bearer "` prefix: `key = auth_header.removeprefix("Bearer ").strip()`. On the Gemini ADK side, change `headers={"x-api-key": MCP_API_KEY}` to `headers={"Authorization": f"Bearer {MCP_API_KEY}"}`. On the Claude Code side, change the `headers` block in `~/.claude.json` to `{"Authorization":

"Bearer sk_live_..."}`. The underlying API key value doesn't need to change — only the header name and format. This is a 3-line change that is backward-compatible if you support both header formats during a transition period.

Q5: In `gemini_client.py`, the results are written to `gemini_results.json` after all three queries complete. If one query's `await run_query()` raises an exception, what happens to the results of the other two queries? How would you improve this?
Answer:
The current code wraps each `run_query()` call in a `try/except` block and appends an error result entry to the `results` list. This means if Q1 raises an exception, Q2 and Q3 still run, and `gemini_results.json` contains three entries — one with `final_answer: "CLIENT EXCEPTION: ..."` and two with actual results. This is correct fail-safe behaviour: one failing query doesn't abort the entire test run. To improve: add a `success: bool` field to each result dict so the comparison script can easily identify which queries failed without string-matching the `final_answer`; and write intermediate results to a file after each query (not just at the end), so a crash during Q3 doesn't lose Q1 and Q2's results.

Q6: Claude Code shows the raw tool result JSON inline in the terminal. Gemini ADK does not show it. From a security perspective, is the Gemini approach actually safer, or does hiding the raw result create a different kind of risk?
Answer:
Hiding the raw result is not safer — it is different. The security risk in showing raw tool results is that if the result contains sensitive data (PII, credentials, internal file paths), any person watching the terminal or reading screen recordings can see it. Gemini's approach of only showing the model's prose summary reduces this accidental exposure. However, it creates a different risk: if the model's summary is incorrect or incomplete (e.g. it silently truncates a large response, or misinterprets an error), the developer cannot tell by looking at the terminal — they must add explicit logging or debugging code to inspect the raw result. The transparent approach (Claude Code's) is better for debugging and auditability; the opaque approach (Gemini's default) is marginally better for accidental data exposure at the terminal. In a production deployment, neither approach should be relied upon for data security — sensitive tool results should be redacted at the server level before they ever leave the server.

Q7: The `read_gcs_object` tool returns content as UTF-8 text, or as hex for binary files. When Claude Code or Gemini ADK passes this response to their respective LLMs, what practical limit does this create, and how would you redesign the tool for real enterprise use with large CSVs?
Answer:

Both Gemini and Claude have context window limits (Gemini 2.0 Flash: ~1M tokens; Claude Sonnet: ~200K tokens). A large file (even within the 10MB GCS_MAX_FILE_SIZE_BYTES limit) can contain hundreds of thousands of tokens, consuming most or all of the context window and either being truncated or causing a context overflow error. For enterprise use with large CSVs the tool should be redesigned to: (a) return only metadata and a preview (first N bytes/rows) by default, with an explicit `full_content: bool` parameter for opt-in full reads; (b) support a `row_range` parameter for CSVs to read specific row ranges; (c) for very large files, return a signed GCS download URL valid for a short time rather than the content itself, letting the user download it directly. The LLM's context window is not a file system — it is a reasoning surface, and you should send only what the model needs to reason about.

Q8: Both clients use streamable-HTTP to connect to the Cloud Run server. How would you connect either client to the Goal 1 server (which uses stdio transport), and why can't you point Claude Code's `~/.claude.json` HTTP config at a stdio server?
Answer:
The Goal 1 server uses stdio — it communicates over the process's stdin/stdout. To connect a client to it via stdio, the client must spawn the server as a subprocess and pipe its stdin/stdout. For Gemini ADK, you'd use `StdioServerParameters(command="python3", args=["path/to/server.py"])` instead of `StreamableHTTPConnectionParams`. For Claude Code, you'd use `"type": "stdio"` in `~/.claude.json` with a `command` and `args` field instead of `"type": "http"` with a `url`. You cannot point the HTTP config at a stdio server because a stdio server has no network socket — it has no URL to point at. A stdio server only exists as a subprocess; it cannot receive HTTP requests. To make a stdio server accessible via HTTP, you'd either (a) use the Goal 2 Cloud Run deployment pattern (wrap it in FastAPI with streamable-HTTP), or (b) use a local proxy tool like `mcp-proxy` that bridges HTTP to stdio.

Q9: The server logs `client_name` on every tool call. If both Gemini ADK and Claude Code use the same API key, the logs don't tell you which client made which call. Design a server-side change that enables per-client attribution without requiring separate API keys.
Answer:
Add a custom HTTP header convention: `X-MCP-Client-Id: <client_identifier>`. Both clients would send this header (e.g., `"X-MCP-Client-Id": "gemini-adk-v0.5"` or `"X-MCP-Client-Id": "claude-code-v1.0"`). In `middleware/auth.py`, read this header and store it: `request.state.client_id = request.headers.get("x-mcp-client-id", "unknown")`. In `middleware/logging.py`, include `client_id` in the structured log entry alongside `client_name`. This is purely additive — the header is optional, the server accepts it if present and uses `"unknown"` if absent. Neither existing client breaks. For audit purposes this is weaker than separate keys (a client can send any `client_id` it wants, so it's attribution, not authentication), but it solves the logging problem without key management overhead.

Q10: If you had to choose one server-side change from Section 5's recommendations to implement today, which one would have the highest impact for a team maintaining this server over 12 months, and why?

Answer:

Recommendation 2 (return `isError: true` in the MCP protocol response) has the highest long-term impact. The current server is only tested against two clients. As the team adds more clients (Cursor, new ADK versions, custom agent loops, third-party MCP-compatible tools), the clients will vary in how they handle tool results. Some will check `result["success"]` in the JSON payload; others will look for `isError` at the protocol level. By implementing spec-compliant `isError: true` now, the server is correctly interpretable by any future MCP client without requiring the server to be updated again. The MCP spec is the contract; Recommendation 2 makes the server honour that contract fully. The other recommendations are good DX improvements, but they are narrow in scope — Recommendation 2 is a correctness fix relative to the protocol spec that future-proofs the server for the entire client ecosystem.